

A.5 - Java Generics

Consider the code for a sequential search through an array of `Integer` objects, returning the index where the object was found (or -1 if not found):

Listing A.5.1.

```
public int search(Integer[] arr, Integer target) {
    int index = 0;
    while (index < arr.length &&
        !arr[index].equals(target) != 0) {
        index++;
    }
    return (index != arr.length) ? index : -1;
}
```

If we want a sequential search through an array of `String`, we need to make a completely new method that has a lot of the same code:

Listing A.5.2.

```
public int search(String[] arr, String target) {
    int index = 0;
    while (index < arr.length &&
        !arr[index].equals(target)) {
        index++;
    }
    return (index != arr.length) ? index : -1;
}
```

There must be a better way. Java solves this problem by using **generic types**. We use a class like this:

Listing A.5.3. The Searcher class

```
class Searcher<T> {
    public int search(T[] arr, T target) {
        int index = 0;
        while (index < arr.length && !arr[index].equals(target)) {
            index++;
        }
        return (index != arr.length) ? index : -1;
    }
}
```

The `<T>` is a **type parameter**; we will fill it in with the type we want when creating a `Searcher` object, as in the following main method:

Listing A.5.4. Using the `Searcher` class

Run

Original - 1 of 1

Show CodeLens

Reformat

```
1 public class SearcherTest {
2     public static void main(String[] args) {
3         Searcher<Integer> searchInt = new Searcher<>();
4         Integer [] arr = {3, 5, 7, 2, 9};
5         int found = searchInt.search(arr, 9);
6         System.out.println("9 found at index " + found);
7
8         Searcher<String> searchStr = new Searcher<>();
9         String[] words = {"cat", "elk", "dog", "fox"};
10        found = searchStr.search(words, "elk");
11        System.out.println("elk found at index " + found);
12    }
13 }
14
```

Thus, just as you fill in a method's parameter between parentheses with an actual argument, you fill in the type parameter between the angle brackets with an actual type name.

All of Java's Collection framework uses type parameters. You can create an `ArrayList` of `Integer`, `String`, `Color`, `HttpRequest`, or any `Object` that you define yourself. All of the `ArrayList` methods will work on these lists, and there is no need for Java to have separate libraries for each data type.

► **Aside**

The only proviso is that the data type you provide *must* be an object type rather than a primitive type. You cannot write `ArrayList<double>`. Instead you must use the object wrapper class: `ArrayList<Double>`.

Sometimes, you also need to specify limitations on the types that can be used for a parameter. For example, if we wanted a `Sorter<T>` class, we could only sort objects that have a `compareTo` method. That is, they must implement the `Comparable<T>` interface. `String` and `Double` have such a method, but `Color` and `HttpRequest` don't.

The beginning of our hypothetical `Sorter` class would look like this:

```
public class Sorter<T extends Comparable<T>> {  
    // code...  
}
```

It is also possible to introduce generic types for the duration of a single method. Presume we wanted a static method named `search` without having to create a `Searcher` class. We could write a program like [Listing A.5.5](#) that uses this technique. The `<T>` specification on line 3 declares the generic type for the `search` method:

Listing A.5.5.

Save & Run



Original - 1 of 1

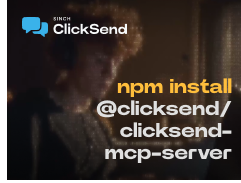
Show CodeLens

Reformat

```
1 public class GenericSearchMethod {
2
3     public static <T> int search(T[] arr, T target) {
4         int index = 0;
5         while (index < arr.length && !arr[index].equals(target))
6             index++;
7     }
8     return (index != arr.length) ? index : -1;
9 }
10
11 public static void main(String[] args) {
12     Integer [] arr = {3, 5, 7, 2, 9};
13     int found = search(arr, 9);
14     System.out.println("9 found at index " + found);
15
16     String[] words = {"cat", "elk", "dog", "fox"};
17     found = search(words, "elk");
18     System.out.println("elk found at index " + found);
19 }
20 }
21
```

There are many other important concepts for using generics. You can see a full tutorial at <https://docs.oracle.com/javase/tutorial/java/generics/index.html>. However, this is a book about data structures rather than the deeper concepts of Java, so we will leave you with this brief introduction to generics, which should be sufficient to help you understand the programs in this book.

You have attempted 1 of 3 activities on this page.



ClickSend MCP Server.

npm install. Your AI can
text now. [Find out
more.](#)

try.clicksend.com

Ads by EthicalAds